Instituto Superior de Formación Técnica Nº 151
Carrera: Analista en Sistemas
3 Año. Algoritmos y Estructuras de Datos III.
Alumna LUZURIAGA Silvana

ARBOLES.

1. Marco Teórico:

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

1.Un grafo ***G*** es una estructura de datos no lineal que representa una red o relacion entre elementos. Matematicamente consta de un par ordenado ***G = (V, E),*** donde:

- **** V(Vertices/Nodos)***: es un conjunto no vacio de puntos o elementos de datos.
- **** E(Aristas/Arcos):*** es un conjunto de lineas o uniones que conectan pares de nodos asociados.

Los grafos pueden ser ***dirigidos***(si las aristas tienen una dirección definida mediante flechas) o ***no dirigidos*** (si las conexiones son bidireccionales y reciprocas).

2. Un **árbol** es una clase restringida de grafo simple, aciclico y conectado. Al no tener ciclos, se caracteriza por poseer una relación jerárquica clara entre sus elementos, donde cada nodo (excepto la raíz) tiene exactamente un predecesor único (padre).

Problemas que resuelve:

* Organización jerárquica de datos: Como la estructura de directorios y archivos de un sistema operativo.

* Optimización de búsquedas: Al estructurar la información ordenadamente, reduce drásticamente el tiempo necesario para localizar datos en bases de datos.

* Evaluación de expresiones: Se usa en compiladores mediante arboles de precedencia aritmética.

*Algoritmos de compresión: Como el árbol de Huffman utilizado en codificación de datos.

* Toma de decisiones y juegos: Modelado de estados futuros (árboles minmax o de decisión).

3. Un árbol binario es una forma especial de árbol donde cada nodo puede tener, como máximo, dos hijos, los cuales se identifican explicitamente como subarbol izquierdo y subarbol derecho. Su función principal es optimizar la organización y manipulación dinámica de datos en memoria para búsquedas eficientes.

Operaciones fundamentales:
*Creación: Agregar un nuevo elemento en la posición adecuada según la lógica del árbol.
*Eliminación: Quitar un nodo especifico reestructurando los enlaces necesarios para mantener las propiedades de la estructura.

*Búsqueda: Localizar eficientemente un valor recorriendo caminos reducidos.
*Recorridos: Visitar secuencialmente todos los nodos una sola vez. Existen tres métodos clásicos recursivos:
- Preorden: Raíz -> Izquierda -> Derecha.
-Inorden: Izquierda -> Raiz ->Derecha.
-Postorden: Izquierda -> Derecha -> Raíz.

4. Un **árbol balanceado** (o árbol AVL) es un árbol binario de búsqueda en la cual, para cada nodo, la diferencia de altura entre su subárbol derecho y su subárbol izquierdo (llamada **factor de equilibrio o FE**) es en valor absoluto menor o igual a 1**(IFEI -< 1)**. Si tras una inserción o eliminación se pierde este equilibrio (FE = 2 o FE =-2), se realizan ajustes estructurales denominados rotaciones (simples o compuestas).

Ejemplos de uso:
*Indexación en motores de bases de datos relacionales en memoria donde se requieren tiempos de respuesta predecibles y óptimos *O (log n).*
*Diccionarios o conjuntos integrados en librerías estándar de lenguajes de programación.

5. Ambas estructuras están diseñadas para gestionar grandes volúmenes de datos guardados en **dispositivos de almacenamiento secundario** (como discos rígidos o SSDS) organizando la información en bloques o “páginas” para minimizar los accesos al disco.

* **Árbol B**: Es un árbol balanceado de búsqueda multi-camino. Cada página almacena datos completos y claves de índice de forma intercalada en todos sus niveles (raíz, páginas intermedias y hojas).
* **Árbol B+**: Es una variante optimizada del Árbol **B.**

Principales diferencias:
* En el **Árbol B+**, **toda la información y los registros reales se almacenan exclusivamente en las páginas hojas**. La raíz y los nodos intermedios solo guardan duplicados de las claves que sirven puramente como indices. En el **Árbol B**, los datos validos están repartidos por todo el árbol.
* Las hojas en un Árbol B+ suelen estar enlazadas de manera secuencial (lista enlazada), lo cual permite hacer recorridos por rangos y consultas secuenciales de manera sumamente eficiente, algo que en el Árbol B tradicional requiere un costoso recorrido de múltiples niveles de paginas.

Marco Practico:

A partir de la siguiente lista proporcionada: **G, B, Q, A, C, K, F, P, D, E, R, H.**

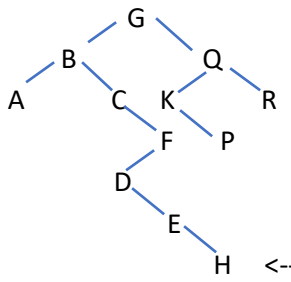
EJERCICIO 1. CONSTRUCCION DEL ARBOL BINARIO DE BUSQUEDA (ABB)

Para construir un ABB, aplicamos la regla: el primer elemento de la raíz, y para los siguientes, si el elemento es menor va a la izquierda y si es mayor va a la derecha(siguiente el orden alfabético).

1. G: Raiz.
2. B: Menor que G -> Izquierda de G.
3. Q: Mayor que G -> Derecha de G.
4. A: Menor que G, menor que B -> Izquierda de B.

5. C: Menor que G, mayor que B -> Derecha de B.
6. K: Mayor que G, menor que Q -> Izquierda de Q.
7. F: Menor que G, mayor que B, mayor que C -> Derecha de C.
8. P: Mayor que G, menor que Q, mayor que K -> Derecha de K.
9. D: Menor que G, mayor que B, mayor que C, menor que F, mayor que D -> Izquierda de F.
10. E: Menor que G, mayor que B, mayor que C, menor que F, mayor que D -> Derecha de D.
11. R: Mayor que G, mayor que Q -> Derecha de Q.
12. H: Mayor que G, menor que Q, menor que K -> Izquierda de K.

Estructura del árbol resultante:

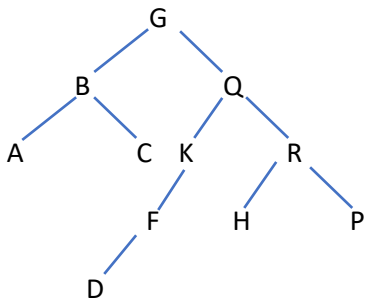


<-- Nota: El elemento H se inserta a la izquierda de K. (H es menor que K). Corrección: K tiene hijos Izq(H) y derecha (P)

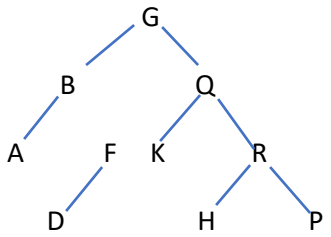
Jerarquias:
* G tiene hijos: Izq(B), Der(Q)
* B tiene hijos: Izq(A), Der(C)
* C tiene hijo: Der(F)
* F tiene hijo: Izq(D)
* D tiene hijo: Der(E)
* Q tiene hijos: Izq(K), Der(R)
* K tiene hijos: Izq(H), Der(P)

EJERCICIO 2. EXTRACCION ITERATIVA DE ELEMENTOS (E, C, G)

A) **Estado tras extraer E:**
El nodo E es un nodo hoja (no tiene hijos), Su eliminación es directa: simplemente removemos el puntero que apuntaba a él desde su padre (D).



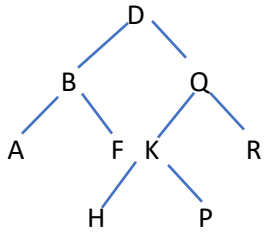
B) **Estado tras extraer C**:
El nodo **C** tiene un solo hijo (el subárbol que empieza en **F**). Para eliminarlo, enlazamos directamente al padre de C (**B**) con el hijo de C (**F**).



C)

D) **Estado tras extraer G (La Raíz)**:
El nodo G tiene dos hijos (B y Q). Según el criterio del marco teórico de tu bibliografía, se debe reemplazar por el elemento mas a la derecha del subárbol izquierdo (el mayor de los menores).
En el subárbol izquierdo de G(que tiene raiz B), el elemento mas a la derecha es el valor maximo disponible, el cual es D.
1. Buscamos el nodo mas a la derecha del subárbol izquierdo: D.
2. Reemplazamos el valor de la raíz por D.
3. Quitamos el nodo original D de su antigua posición.

Árbol Final tras todas las extracciones:



EJERCICIO 3. ALGORITMOS DE RECORRIDO(SOBRE EL ÁRBOL ORIGINAL DEL EJERCICIO 1)
Aplicando las reglas del recorrido sobre el diseño original construido en el Ejercicio 1:
a) Recorrido en Preorden (Raíz - Izquierda - Derecha)
Avanza verticalmente registrando el nodo apenas se visita:
* Resultado: G, B, A, C, F, D, E, Q, K, H, P, R.
b) Recorrido en Inorden (Izquierda - Raíz - Derecha)
Procesa los elementos de izquierda a derecha. En un ABB, **este recorrido siempre da la lista ordenada alfabéticamente**:
* Resultado: A, B, C, D, E, F, G, H, K, P, Q, R.
c) Recorrido en Postorden (Izquierda - Derecha - Raiz)
Procesa los subárboles descendientes por completo antes de listar los nodos padres o raíces correspondientes:
* Resultado: A, E, D, F, C, B, H, P, K, R, Q, G